Microservices Architecture

Software Architecture Notes — Knowledge Base

1. What is it?

Microservices is an architectural style where an application is built as a collection of small, independently deployable services, each responsible for a single business capability. Services communicate over the network, typically via HTTP/REST, gRPC, or asynchronous messaging, rather than through in-process function calls.

2. Core Characteristics

- Each service owns its own data store (database-per-service), avoiding shared databases.
- Services are independently deployable — a change to one service does not require redeploying others.
- Organized around business capabilities rather than technical layers.
- Decentralized governance — teams can choose different languages/stacks per service (polyglot).
- Failure isolation — a failure in one service should not cascade and take down the whole system.

3. Communication Patterns

Services talk to each other synchronously (REST, gRPC) or asynchronously (message queues, event streams like Kafka/RabbitMQ). Asynchronous, event-driven communication reduces tight coupling and improves resilience, at the cost of eventual consistency.

4. Advantages

- Independent scaling — scale only the services under load.
- Faster, safer deployments through smaller, isolated codebases.
- Technology flexibility per service.
- Better fault isolation compared to a single monolithic process.

5. Challenges

- Distributed systems complexity: network latency, partial failures, retries.
- Data consistency across services (no single ACID transaction spanning services).
- Operational overhead: needs strong DevOps, observability (logging, tracing, metrics), and automation.
- Testing is harder — requires contract testing and integration environments.

6. Common Supporting Patterns

- API Gateway — single entry point that routes requests to services.
- Service Discovery — services find each other dynamically (e.g., Consul, Eureka).
- Circuit Breaker — prevents cascading failures (e.g., Hystrix, resilience4j).
- Saga Pattern — manages distributed transactions via a sequence of local transactions with compensations.

7. When to Use

Best suited for large, complex applications with multiple teams that need independent release cycles and scaling. For small applications or early-stage products, the operational overhead often outweighs the benefits — a monolith is usually the better starting point.